Component A

Direct quotes

- “Students are live editing an Excel sheet that's on the shared drive.”
- “I want everything to automate. I don't care how it happens, but then I would love to see all of it.”
- “I need it for the entire quarter. I don't delete it… it just gets put into archive”

Surprises

- Dorothy may handle 20–150 orders per month.
- Even form tools like Google Forms still resulted in blank or incomplete submissions.

Current workarounds

- Shared Excel sheets with multiple tabs
- Students manually enter purchase requests into the Excel sheet, including information like team number, CFO, provider/supplier, quantity, name of team, price, link to purchase, notes, approval from instructor
- Dorothy locks the sheet after a submission deadline
- Manual approval process with instructor
- Communication with teams (mainly CFO) through follow-up messages
- Manual handling of refunds and updating team budgets
- Excel-based summary and tracking system
  Dorothy uses multiple tabs in Excel where data from different sheets is automatically aggregated into a final summary tab. This allows her to track total spending per team and maintain an overall view of all purchase requests, but it is still difficult to manage and not user-friendly for students.
- Tracking orders and receipts through: Amazon business account, supplier websites, emails, physical packing slips

What success looks like

- Easier submission for students
- Fewer incomplete requests
- Less back-and-forth
- Better visibility into request status, approvals, spending, and receipts

User journey

1. A student team decides they need to buy an item.
2. The team / CFO gathers purchase information such as team number, course number, supplier, item name, quantity, unit price, total price, link, and notes.
3. The student enters the request into a shared Excel sheet on the shared drive.（What could go wrong: wrong item, missing specs, missing link, wrong course/team info）
4. Dorothy checks the sheet after the submission deadline.

5. Dorothy reviews whether the request is complete and whether all required information is included.
6. Dorothy works with the instructor to get approval for the purchase.
7. If information is missing, wrong, or unclear, Dorothy follows up with the team CFO or multiple team members.
8. Dorothy places the order, often once per week for non-Amazon purchases.
9. Dorothy tracks the shipping / arrival status using Amazon, supplier accounts, emails, and physical packing slips.
10. When the item arrives, she verifies the package and keeps the receipt / packing slip until it is verified in Workday.
11. Dorothy updates the Excel sheet so spending totals and team budgets remain accurate.
12. If an order is delayed, lost, backordered, or refunded, Dorothy manually updates the records and communicates with the student team.

Problem statement

When Dorothy needs to manage student purchase requests, she currently relies on a shared Excel sheet and manual follow-up communication, which causes incomplete data, inefficient tracking, and a heavy administrative workload.

Component B

AI Usage Log

Interaction 1

Prompt

Build a functional Streamlit app to help Dorothy manage student purchase requests, replacing a shared Excel workflow.

The goal is to reduce incomplete submissions, minimize back-and-forth communication, and provide a centralized view of all requests and team budgets.

The app should include:

- A structured student submission form with validation
- A dashboard for Dorothy to view, filter, and update request status
- Persistent data storage using SQLite
- A budget summary per team
- An archive view for historical data

It should enforce required fields, prevent incomplete submissions, and support status tracking (Submitted, Approved, Ordered, Delivered, Refunded, Archived). No external APIs should be used.

What the AI produced

The AI generated a full implementation plan and corresponding code for a Streamlit app. It created:

- A working multi-page structure:
    - Submission form page
    - Dashboard page with filtering and status updates
    - Budget summary page with total spending calculation
    - Archive page
- A SQLite-based data storage system with tables for requests and budgets
- Form validation logic (required fields, numeric checks, link validation)
- Status tracking system with predefined statuses
- Automatic calculation of total price and team budget summaries

The app was runnable locally and covered most of the core functionality defined in the prompt.

Did it work on the first try? What did you fix?

The app worked on the first try in terms of basic functionality and could run successfully. Most core features were implemented correctly.

However, there were several issues:

- The app did not clearly separate user roles (students vs. Dorothy), making the interface less intuitive
- The UI was not very user-friendly and felt more like a technical prototype
- Navigation and interaction design could be improved for clarity

To address these issues, I continued iterating by prompting the AI to:

1. Separate student and admin (Dorothy) views
2. Improve usability and interface clarity
3. Refine the interaction flow

Interaction 2

Prompt

I asked the AI to refine the app with two main improvements:

1. Simplify budget management:
   Instead of manually setting budgets per team, Dorothy should define budgets at the

course project level, and students select the corresponding project when submitting requests. The system should then track budget spending automatically.

2. Add a login system with role-based views:

- Users log in with email
- First-time users choose a role (student or admin)
- Students (@uw.edu) can only:
    - submit purchase requests
    - view their own requests and status
- Admins (Dorothy) can access:
    - dashboard
    - budget tracking
    - archive
    - project setup

What the AI produced

The AI successfully implemented:

- A login system with email and role selection
- Role-based views (student vs. admin)
- A user table to store email and role
- Student view:
    - submission form
    - “My Orders” page showing their requests
- Admin view:
    - dashboard
    - project setup page
    - budget tracking page

Did it work on the first try? What did you fix?

The login system and role-based access worked correctly on the first try.

However, the AI misunderstood the budget requirement:

- It treated the budget as a single course-level budget,
- instead of tracking spending per team within a course/project.

To fix this, I clarified that:

- each team still needs its own budget tracking
- the course/project only provides a predefined budget template

I then continued iterating to correct the budget logic.

Interaction 3

Prompt

I clarified the budget tracking logic because the previous implementation was incorrect.

The budget should not be tracked at the course level, but at the team level within each course/project.

Each course project has a predefined per-team budget, and the system should:

- track spending per team (not per course)
- group data by (course, team)
- calculate total spent per team
- subtract refunded requests
- compute remaining budget for each team

What the AI produced

The AI updated the data logic and implementation correctly:

- Refactored the budget function from project-level to team-level aggregation
- Updated SQL query to group by both course and team
- Calculated total spending per team:
    - excluding refunded requests
    - excluding archived data
- Joined with the course_projects table to retrieve the per-team budget
- Computed remaining budget for each team
- Updated the budget tracking page to display:
    - project
    - team
    - per-team budget
    - total spent
    - remaining budget

Did it work on the first try? What did you fix?

The initial implementation had a misunderstanding:

- It treated budget as a single course-level value, instead of per-team tracking

After clarifying the requirement, the AI corrected:

- the data aggregation logic
- the SQL grouping (course + team)
- the budget calculation

The updated version worked correctly and aligned with the real workflow.

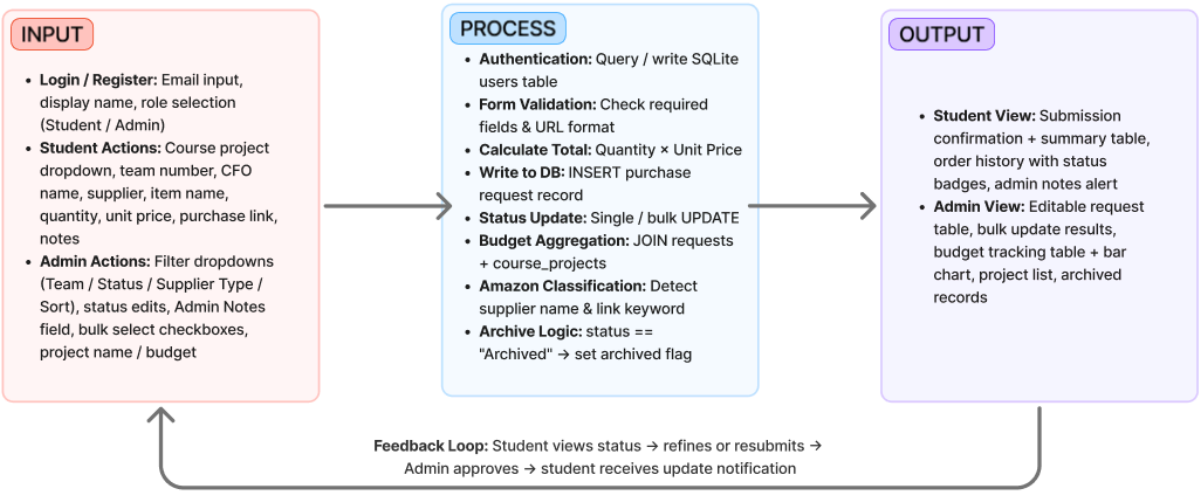
Accessibility baseline results

Color contrast check result: PASS

Semantic headings check result: PASS

Component C

Architecture diagram



Design Decision Log

Field	Your Entry
Decision	What did you decide? Keep all code in a single `app.py` file (1,127 lines), rather than splitting into multiple modules such as `db.py`, `pages/student.py`, `pages/admin.py`, or `utils.py`.
Alternatives considered	What else could you have done? 1. Multi-file structure: separate `db.py` (database helpers), `validation.py`, `pages/student.py`, `pages/admin.py`. 2. Streamlit native multi-page: place each page as a separate file inside a `pages/` folder, which Streamlit supports natively.
Why you chose this	What constraint drove it? The course rule (`.cursor/rules`) explicitly required: *"Keep all code in app.py (single file only)"*. Beyond the rule, a single file eliminates cross-file import complexity, is easier

	to share and run in a demo environment, and Cursor AI also followed this constraint when generating the code.
Trade-off	What did you give up? Gave up: modularity, independent testability of each layer (DB / validation / UI), easier parallel collaboration across files. Gained: zero import overhead, one-file portability, simpler mental model for a course prototype.
When would you choose differently?	Under what conditions? If the app grew past ~800 lines with more features (e.g. notifications, file uploads, export), or if a team needed to work on different parts simultaneously, or if unit testing was required — at that point splitting into `db.py` + `pages/` would clearly pay off.

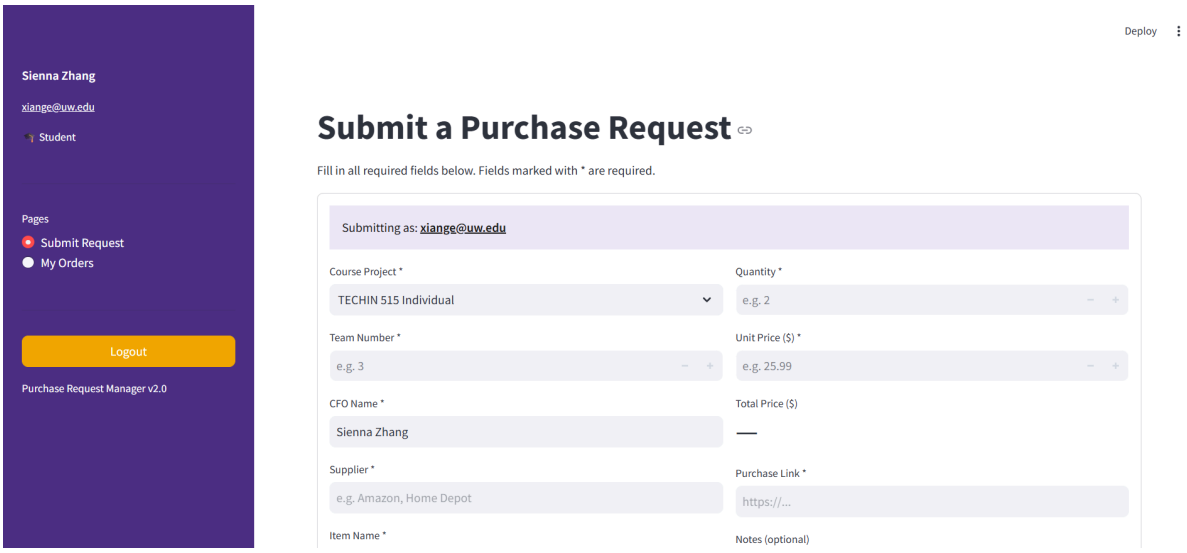
Component D

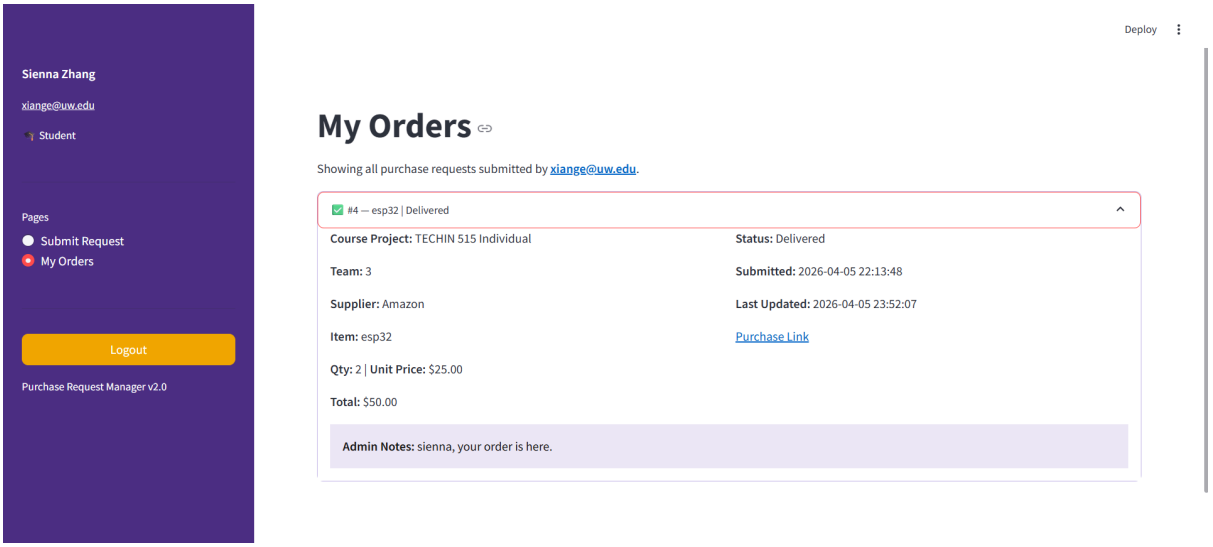
Validation results (the completed smoke-test table)

Feature Tested	Action You Took	Expected Result	Actual Result	Pass/Fail
Purchase Request Submission Form	Filled in all required fields (team number, supplier, item name, quantity, unit price, purchase link) and clicked "Submit Request"	Form validates inputs, calculates total price automatically (quantity × unit price), saves record to SQLite DB, and displays a confirmation message with a summary table	Showed "Request #[ID] submitted successfully!" with a full summary table; total price was auto-calculated correctly; form cleared after submission	*Pass*
Admin Dashboard — Filter by Status	On the Dashboard page, selected "Approved" from the "Filter by Status" dropdown	Table should refresh to show only requests with status = "Approved", hiding all other statuses	Table updated immediately to display only Approved requests; row count in "Showing X request(s)" updated accordingly	*Pass*

Budget Tracking — Metric Cards + Bar Chart	Navigated to the "Budget Tracking" page after multiple purchase requests had been submitted	Page should display a per-team spending table with color-coded "Remaining ($)" column (green/orange/red), plus three summary metric cards (Total Allocated / Total Spent / Total Remaining) and a grouped bar chart	Table rendered with correct budget vs. spent values; remaining balance shown in green (>$50); metric cards displayed correct totals; bar chart grouped by Project — Team label in UW purple and gold	*Pass*

Screenshot of your running app showing at least one tested feature





Dorothy

dorothy@uw.edu

Admin

Pages

- Dashboard
- Course Project Setup
- Budget Tracking
- Archive

Logout

Purchase Request Manager v2.0

Deploy

Dorothy's Dashboard

View, filter, and update the status of all active purchase requests.

Filter by Team: All

Filter by Status: Submitted

Filter by Supplier Type: All

Sort by Date: Newest first

Showing **2** request(s)

Select	ID	Submitter	Team	CFO	Course	Supplier	Supplier Type	Item	Qty	Unit Price ($)	Total ($)	Status
☐	6	polly@uw.edu	5	Polly Yao	TECHIN 515 Team	Amazon	Amazon	rasberry pi	1	45	45	Submitted
☐	5	polly@uw.edu	4	Polly Yao	TECHIN 515 Individual	digikey	Non-Amazon	FSR	5	3	15	Submitted

Bulk Status Update

Tick the **Select** checkboxes in the table above, then choose a target status and click Apply.

Target Status for Selected Rows: Submitted

Apply to Selected

Dorothy

dorothy@uw.edu

Admin

Pages

- Dashboard
- Course Project Setup
- Budget Tracking
- Archive

Logout

Purchase Request Manager v2.0

Deploy

Budget Tracking

Each course project has a **per-team budget**. This page tracks how much each team has spent within their project.

Project	Team	Per-Team Budget ($)	Spent ($)	Remaining ($)
TECHIN 515 Individual	3	$70.00	$50.00	$20.00
TECHIN 515 Individual	4	$70.00	$15.00	$55.00
TECHIN 515 Team	5	$350.00	$45.00	$305.00
TECHIN515	1	$0.00	$7.98	$-7.98

Total Allocated: $490.00

Total Spent: $117.98

Total Remaining: $372.02

Spending by Team

Dorothy

dorothy@uw.edu

Admin

Pages

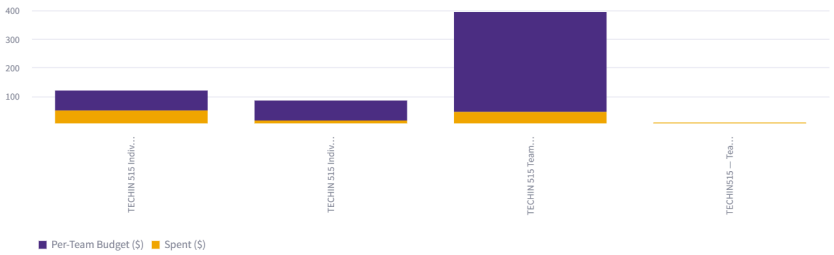
- Dashboard
- Course Project Setup
- Budget Tracking
- Archive

Logout

Purchase Request Manager v2.0

Deploy

Spending by Team

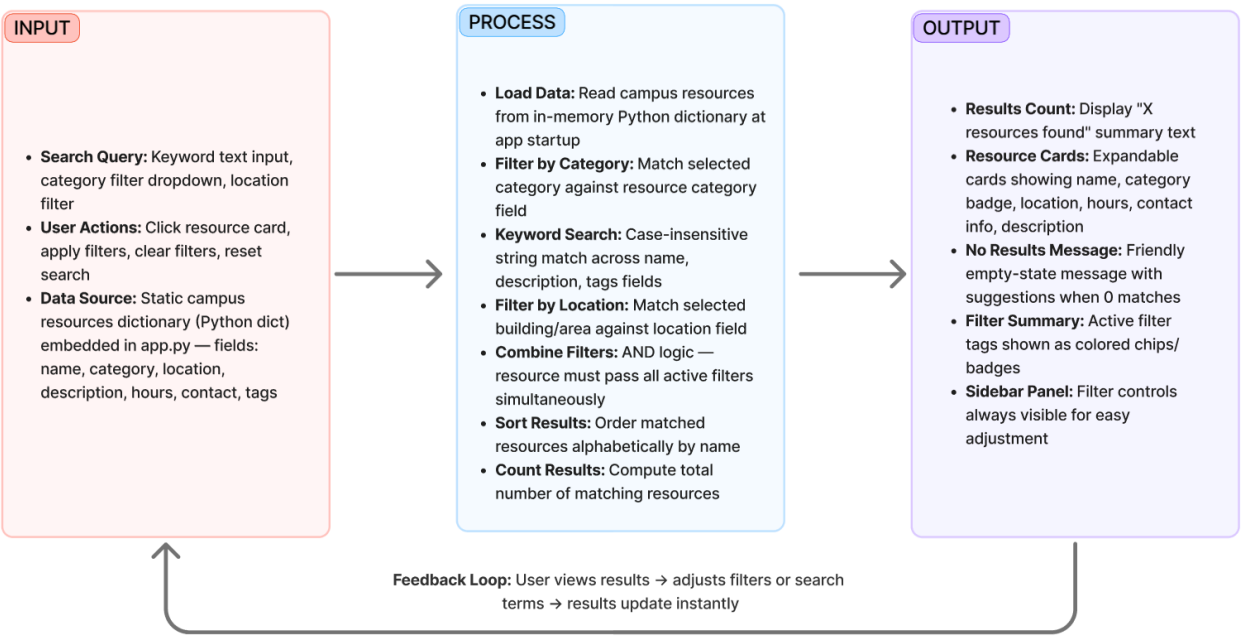


Quality gate checklist (completed)

- ☑ Smoke test table completed with 3 tested features
- ☑ Any failed test is either fixed and re-tested, or clearly documented
- ☑ Screenshot of the running app included
- ☑ Accessibility baseline results recorded (color contrast + semantic headings)

Component E

I/P/O diagram for the Wayfinder app



Edge-case validation notes for 2 edge cases

Edge case A: Strict combined filters yield zero main results, but the keyword still triggers “related suggestions”

Scenario: Keyword `coffee`, category “All”, location “GIX Building - 3rd Floor”. The main result list is empty (no dining on the 3rd floor; Kitchen is on the 2nd, vending on the 1st).

Why validate this: If you only test “happy paths” with matches, you can miss the empty state when **AND logic** stacks with **floor** filters. New students often search by floor but still need to be guided to dining-related resources. Checking this ensures: (1) the empty-state

message still appears; (2) the “Food & drink nearby” block still lists Dining resources and is **not** suppressed by the location filter, so users do not assume “there is no coffee/food on campus.”

Result: Run the app with the inputs above; confirm the main area shows 0 resources and that Kitchen / Vending Machine (etc.) still appear in the suggestion section below.

Edge case B: Whitespace-only or empty keyword

Scenario: The keyword field contains only spaces, or is left empty, possibly combined with category and/or location filters.

Why validate this: `filter_resources` uses `search_query.strip()` to decide whether the keyword filter applies. Without testing this, it is easy to misread whether spaces count as a search term, or to skip the path where **no keyword** means “filter only by category + location.” Validation keeps filter semantics clear and avoids surprising empty results or skipped location filters.

Result: Leave the keyword empty or use spaces only; set category to “Dining” and location to “All”—all dining resources list consistently. Whitespace-only input is not treated as a non-empty search string.

One `assert` statement for data integrity (include it in code and mention where)

Location: In `[app.py](app.py)`, immediately after the `CAMPUS_RESOURCES` and `CATEGORY_COLORS` constants and **before** the helper functions (e.g. `get_all_categories`).

Code (single `assert` statement):

```
_REQUIRED_RESOURCE_KEYS = frozenset(
    {"name", "category", "location", "description", "hours", "contact", "tags"}
)
assert (
    all(_REQUIRED_RESOURCE_KEYS <= r.keys() for r in CAMPUS_RESOURCES)
    and all(isinstance(r["tags"], list) for r in CAMPUS_RESOURCES)
    and all(r["category"] in CATEGORY_COLORS for r in CAMPUS_RESOURCES)
), "Each campus resource must include required keys, list-typed tags, and a category key in CATEGORY_COLORS."
```

Purpose: At startup, the app checks the static data: every record has the expected fields (avoids `KeyError`), `tags` is a list (because `matches_search` iterates tags), and every `category` has an entry in `CATEGORY_COLORS` (avoids inconsistent UI fallback colors). If any check fails, an `AssertionError` is raised immediately so data-entry mistakes are caught early.

Prompt log excerpt:

Initial prompt

> Build the GIX Campus Wayfinder from the materials and plan we were given: Streamlit, static Python dicts for data, search and filtering, clear result display, and handling when nothing matches.

Refinement

> When I search for “coffee” I get zero resources. Can you add recommendations for dining-related places after that kind of search? New students do not know what resources exist yet, so please suggest nearby or related options.

Short note: what changed and why

Item	Explanation
Change	Extended beyond “filters + display as in the plan” so that when the main filter set returns zero rows **and** the keyword hits preset triggers (e.g. coffee, food), the app shows an extra block of **related-category** suggestions (e.g. **Dining**).
Reason	Under the original plan, keywords only matched name/description/tags; with a floor selected, users can get “semantically related but zero rows under current filters.” New students do not know resource names yet, so intent-based suggestions are needed instead of only an empty state.

Reflection Deliverables

1. What surprised you about AI-assisted coding?
What surprised me most was the speed at which AI could scaffold a fully functional application. I went from a rough idea to a working Streamlit app with SQLite database, user

authentication, form validation, budget tracking, and an admin dashboard in a fraction of the time it would have taken me to write from scratch.
The "wow" moment was when Cursor generated the entire database schema and CRUD operations in one pass, and the app just worked on the first run. On the other hand, it was harder than expected to get the UI details right — things like sidebar styling with UW Purple and GIX Gold required multiple rounds of back-and-forth prompting to match what I had in mind.

2. What did the AI get wrong?
The AI made several mistakes that I had to fix manually. For example, in the budget tracking page, the AI initially calculated budgets at the project level rather than per-team, which didn't match Dorothy's actual workflow — each team within a project gets its own budget. I caught this during testing and had to rewrite the get_budget_by_team() function to join spending data correctly against per-team budgets. This experience taught me that AI-generated code still needs careful review and testing.

3. Could you explain your code?
Yes. For example, the validate_form() function (around line 380–409 in app.py) takes in all the fields from the student submission form — team number, CFO name, course project, supplier, item name, quantity, unit price, and purchase link — and checks each one against specific rules. It builds up a list of error strings: if the team number is missing or not positive, it appends an error; if the CFO name is blank, it appends another; and so on. For the purchase link, it first checks if it's empty, and only then checks if it starts with "http" — avoiding a redundant error. The function returns the full list of errors, which the caller uses to decide whether to show error messages or proceed with the database insert. I could walk a classmate through this line by line and explain why each validation exists.

4. What did you learn from the interview?
Talking to Dorothy completely reshaped what I built. Before the interview, I assumed the tool just needed a simple form for students to request purchases and a list view for the admin. But Dorothy explained that she manages purchasing across multiple course projects, each with different per-team budgets, and that she needs to distinguish Amazon orders from non-Amazon orders because they follow different procurement workflows. She also mentioned that she often updates many requests at once — for example, marking a batch as "Ordered" after placing a bulk purchase. These insights led me to add the course project setup page, per-team budget tracking with visual charts, the Amazon/Non-Amazon classification, and the bulk status update feature on the dashboard — none of which I would have thought to include without that conversation.